

1. Описание системы

1.1. Полное, сокращенное наименование и условное обозначение системы.

- **Полное наименование системы:** Система активной сетевой защиты и автоматического реагирования на инциденты на базе ИИ-агента;
- **Сокращенное наименование:** САЗИИ (Система Активной Сетевой Защиты с ИИ);
- **Условное обозначение (кодировое имя проекта):** AI-Sentinel (или Aegis-Agent).

1.2. Цель и назначение системы.

Цель создания системы: Повышение эффективности обнаружения и нейтрализации сетевых кибератак за счет автоматизации процессов триажа инцидентов автономным ИИ-агентом, а также исследование устойчивости такого агента к состязательным атакам (Prompt Injection) на уровне сетевых интерфейсов.

Назначение системы:

- Непрерывный мониторинг входящего и внутрисетевого трафика распределенной микросервисной инфраструктуры;
- Инспекция сетевых аномалий и логов ловушек (Honeynet) с использованием языковых моделей (LLM);
- Автоматизированное принятие решений на сетевом уровне (динамическая блокировка IP-адресов атакующих через механизмы iptables/nftables);

- Регистрация инцидентов безопасности в системах учета уязвимостей (DefectDojo) с целью снижения нагрузки на аналитиков (SOC L1/L2) и минимизации времени реагирования (MTTR).

1.3. Связь с другими системами и IT проектами

Разрабатываемая система интегрируется со следующими внешними и внутренними ИТ-компонентами:

- **Системы виртуализации и контейнеризации (Docker, Kubernetes/K3s):** Изолированная среда выполнения целевых микросервисов и защищаемого периметра;
- **Системы контроля версий и автоматизации CI/CD (облачная платформа GitHub Actions):** Среда автоматизированной сборки, тестирования (SAST/SCA/DAST) и деплоя компонентов системы (GitOps-подход);;
- **Системы сбора телеметрии и логов (Fluent Bit):** Компонент агрегации сырых сетевых логов и дампов трафика для их последующей передачи ИИ-агенту;
- **Платформы управления уязвимостями (DefectDojo):** Внешняя система регистрации подтвержденных инцидентов безопасности и формирования аудиторских отчетов;
- **Внешние API языковых моделей (LLM APIs / Ollama для локального запуска):** Когнитивное ядро ИИ-агента для семантического анализа атак и вынесения вердиктов.

1.4. Плановые сроки начала и окончания работ по созданию системы

С учетом того, что старт работ назначен на **20 сентября**, весь жизненный цикл разработки и исследования разбит на 6 этапов общей продолжительностью около 4 месяцев (сентябрь — январь):

№ Этапа	Наименование этапа	Плановые сроки	Основные результаты
Этап 1	Проектирование архитектуры и моделирование угроз	20 сентября — 10 октября	Разработка схемы сети, DFD-диаграмм, матрицы угроз (STRIDE/PASTA), подготовка репозитория на GitHub
Этап 2	Развертывание инфраструктуры и Honeynet-стенда	11 октября — 31 октября	Настройка Docker/K3s через Ansible, развертывание боевых сервисов и ловушек (Cowie), настройка Fluent Bit
Этап 3	Разработка когнитивного ядра ИИ-агента и интеграций	1 ноября — 25 ноября	Написание логики агента на Python, интеграция с LLM, реализация механизма промпт-стражников (Prompt Guardrails)
Этап 4	Реализация автоматического реагирования (SOAR) и DefectDojo	26 ноября — 15 декабря	Настройка динамической блокировки трафика (iptables), интеграция с REST API DefectDojo, настройка CI/CD пайплайна (SAST/SCA)
Этап 5	Экспериментальное исследование и тестирование (Research)	16 декабря — 10 января	Проведение экспериментов (симуляция атак, оценка точности ИИ-агента, тестирование устойчивости к Prompt Injection)
Этап 6	Оформление результатов и подготовка отчетности	11 января — 31 января	Написание пояснительной записки, оформление графического материала, подготовка презентации к защите

2. Технические и функциональные требования

2.1. Описание технического процесса (Архитектура и компоненты системы)

Система построена на принципах микросервисной архитектуры и контейнеризации. Технический процесс функционирования системы обеспечивается взаимодействием следующих основных компонентов:

2.1.1. Модуль имитации сетевого периметра (Honeynet & Targets)

- **Назначение:** Создание изолированной виртуальной среды для привлечения и перехвата сетевых атак.
- **Состав:**
 - **Фронтенд-ловушки (Decoys):** Эмуляторы уязвимых сервисов (например, программная ловушка Cowrie для эмуляции протоколов SSH/Telnet).
 - **Целевой микросервис (Target App):** Искусственно уязвимое веб-приложение (на базе FastAPI/Flask) с открытыми эндпоинтами для тестирования DAST-сканерами.
- **Технологии:** Docker, Docker Compose, виртуальные сетевые мосты (Docker Networks).

2.1.2. Модуль сбора сетевой телеметрии (Telemetry & Log Ingestion)

- **Назначение:** Непрерывный сбор, агрегация и первичная нормализация логов безопасности, сетевых потоков и системных вызовов.
- **Состав:**

- Агент сбора данных (Fluent Bit), развернутый как демон на хосте и внутри контейнеров.
- Буферные очереди для предотвращения потери пакетов при пиковых нагрузках.
- **Технологии:** Fluent Bit, JSON-форматирование логов, системные сокеты Docker.

2.1.3. Когнитивное ядро ИИ-агента (AI Agent Core / Backend)

- **Назначение:** Анализ поступающих потоков событий в реальном времени, семантическая классификация угроз, принятие решений об изоляции и формирование отчетов.
- **Состав:**
 - **Модуль приема вебхуков (API Gateway):** Принимает нормализованные логи от сборщика.
 - **Аналитический модуль (LLM Client):** Отправляет контекст атаки в языковую модель с инструкциями по классификации.
 - **Модуль защиты от инъекций (Prompt Guard):** Фильтрует входящие данные на предмет состязательных атак (Prompt Injection) против самого агента.
- **Технологии:** Python 3.10+, FastAPI, AsyncIO, интеграция с LLM API (или локальной Ollama).

2.1.4. Модуль активного реагирования и сетевого контроля (SOAR & Firewall Enforcer)

- **Назначение:** Непосредственное исполнение команд по блокировке сетевого трафика и взаимодействию со сторонними системами.
- **Состав:**

- **Фаервол-адаптер:** Интерфейс взаимодействия с межсетевым экраном хоста (iptables / nftables).
- **Интеграционный адаптер DefectDojo:** Клиент для отправки структурированных находок и отчетов об инцидентах в систему учета уязвимостей.
- **Технологии:** Python Subprocess (для управления iptables), REST API клиента DefectDojo.

2.1.5. Модуль автоматизированного тестирования и CI/CD (GitHub Actions Pipeline)

- **Назначение:** Автоматизация процессов сборки, статического анализа кода, сканирования образов и проверки устойчивости системы.
- **Состав:**
 - **Статический анализатор (SAST):** Проверка исходного кода агента и микросервисов (SonarQube / Semgrep).
 - **Сканер контейнеров (SCA):** Проверка Docker-образов на известные уязвимости (Trivy).
 - **Модуль симуляции атак (Chaos/Red Team Runner):** Автоматический скрипт генерации тестовых атак для проверки срабатывания ИИ-агента.
- **Технологии:** GitHub Actions, Trivy, Semgrep.

2.2. Описание бизнес-процесса (Функциональные сценарии работы системы)

Бизнес-процесс функционирования системы описывает сценарии взаимодействия компонентов от момента фиксации аномалии до генерации отчетности и проведения исследовательских тестов.

2.2.1. Сценарий непрерывного мониторинга и сбора событий

- **Инициализация:** Все компоненты системы (ловушки, бэкенд, шлюзы) запущены и функционируют в изолированной среде Docker.
- **Процесс:**
 1. Агент сбора телеметрии (**Fluent Bit**) непрерывно отслеживает файлы журналов (/var/log/, stdout контейнеров).
 2. При возникновении события безопасности (например, попытка несанкционированного входа в ловушку SSH) событие мгновенно сериализуется в структурированный JSON-формат.
 3. JSON-пакет передается по внутреннему защищенному протоколу на входной API-шлюз ИИ-агента.

2.2.2. Сценарий интеллектуального анализа и триажа инцидента

- **Инициализация:** ИИ-агент принимает входящий пакет логов от модуля сбора телеметрии.
- **Процесс:**
 1. **Модуль защиты от инъекций (Prompt Guard)** проверяет текст логов на наличие вредоносных команд (попыток внедрения промптов — Prompt Injection).
 2. Если обнаружена инъекция, запрос изолируется, а в систему мониторинга отправляется метрика о попытке атаки на сам ИИ.
 3. Если данные чисты, когнитивное ядро ИИ-агента анализирует контекст: сопоставляет IP-адрес, частоту запросов и характер введенных команд с матрицей тактик MITRE ATT&CK.
 4. Агент выносит вердикт о степени критичности инцидента (Critical / High / Medium / Low).

2.2.3. Сценарий автоматического активного реагирования (SOAR)

- **Инициализация:** ИИ-агент вынес вердикт о наличии подтвержденной кибератаки (критичность High или Critical).
- **Процесс:**
 - Агент формирует управляющую команду для сетевого экрана хоста.
 - Модуль активного реагирования применяет правило блокировки на уровне сетевого стека (`iptables -I INPUT -s <attacker_ip> -j DROP`). Сетевой доступ атакующего к инфраструктуре мгновенно прекращается.
 - Параллельно агент обращается по REST API к платформе DefectDojo, где автоматически создает тикет (Finding) с подробным описанием атаки, таймлайном и доказательствами (Evidence).

2.2.4. Сценарий автоматизированного CI/CD тестирования безопасности (Research & Verification)

- **Инициализация:** Разработчик отправляет изменения (коммит) в репозиторий проекта на GitHub.
- **Процесс:**
 1. Срабатывает триггер GitHub Actions, запускающий конвейер сборки (CI/CD Pipeline).
 2. На первом этапе SAST (Semgrep) и SCA (Trivy) проверяют код и зависимости на наличие уязвимостей. Если найдены критические баги, сборка прерывается.
 3. На этапе деплоя инфраструктура автоматически разворачивается на тестовом стенде.

4. Запускается модуль Chaos/Red Team Runner, который симулирует серию атак для проверки точности срабатывания ИИ-агента и замеряет метрики задержки (Latency) и точности (Accuracy).

5. Отчет о прохождении тестов компилируется и прикрепляется к коммиту.

3. Техническое решение

3.1. Общее описание технического решения

Разрабатываемая Система Активной Сетевой Защиты с ИИ (САЗИИ) представляет собой аппаратно-программный комплекс эшелонированной обороны, сочетающий принципы активной обороны (Deception Technology / Honeynet), непрерывного мониторинга логов и автоматизированного реагирования на инциденты (SOAR) под управлением когнитивного ИИ-агента.

Архитектурное решение спроектировано по принципу разделения сред: боевая инфраструктура изолирована от ловушек, предназначенных исключительно для раннего выявления злоумышленников, сбора цифровых отпечатков (Fingerprints) атакующих и нейтрализации угроз до их переноса на критические узлы. Управление жизненным циклом системы, сборка компонентов и проверка их безопасности полностью автоматизированы с использованием концепции Infrastructure as Code и конвейера CI/CD на базе **GitHub Actions**.

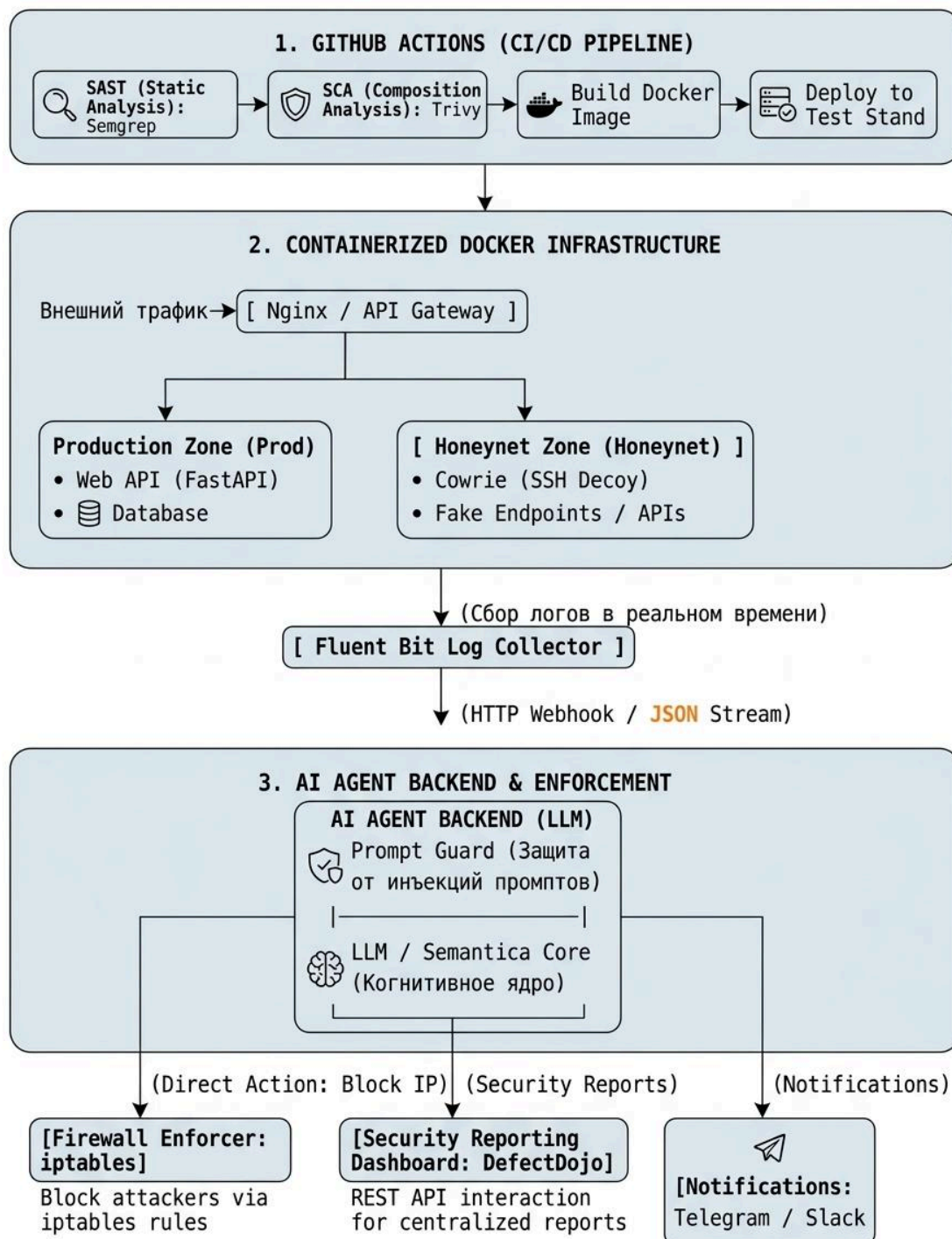


Рисунок 1 — Структурная схема системы (представлена в виде текстово-графического блока)

3.1.1. Состав компонентов системы

Программно-аппаратный комплекс системы состоит из следующих функциональных модулей и подсистем:

1. Подсистема сетевой инфраструктуры и ловушек (Honeynet Infrastructure):

- Компонент эмуляции сервисов (Cowrie): Высокоинтерактивная ловушка, эмулирующая уязвимые службы SSH и Telnet для перехвата брутфорс-атак и записи команд злоумышленников.
- Целевой микросервис (Target Web Application): Тестовое веб-приложение на базе фреймворка FastAPI, содержащее преднамеренные уязвимости для проведения DAST-тестирования.
- Сетевые шлюзы (Nginx / Docker Bridge): Обеспечивают изоляцию сетей (prod_net и honeynet) и маршрутизацию входящего трафика.

2. Подсистема сбора и нормализации телеметрии (Telemetry & Ingestion):

- Агент сбора логов (Fluent Bit): Легковесный демон, выполняющий непрерывный сбор системных журналов хоста, логов доступа Nginx, JSON-логов веб-приложений и сессионных логов ловушек Cowrie.
- Нормализатор данных: Внутренний компонент первичной очистки, приводящий разнородные текстовые логи к единому структурированному JSON-формату.

3. Когнитивное ядро ИИ-агента (AI Agent Core):

- Модуль защиты от состязательных атак (Prompt Guard): Фильтр предварительной проверки входящего текста на наличие инструкций по обходу защиты (Prompt Injection), предотвращающий манипуляцию поведением языковой модели.
- Аналитический модуль (LLM Client & Triage Engine): Асинхронный сервис на Python, осуществляющий семантический анализ нормализованных логов, сопоставление их с матрицей тактик MITRE ATT&CK и вынесение вердикта о критичности инцидента.

4. Подсистема активного реагирования (SOAR & Enforcer):

- Сетевой адаптер (Firewall Enforcer): Модуль взаимодействия с межсетевым экраном ядра Linux (iptables/nftables), выполняющий динамическое добавление IP-адресов атакующих в списки блокировки (DROP).
- Интеграционный модуль DefectDojo: Клиент REST API для автоматического создания тикетов безопасности, фиксации находок (Findings) и выгрузки отчетов об инцидентах.

5. Подсистема автоматизации и контроля качества (CI/CD Pipeline):

- Конвейер GitHub Actions: Автоматизированная среда сборки, тестирования и развертывания проекта.
- Модули сканирования (SAST/SCA): Инструменты Semgrep и Trivy, встроенные в пайплайн для поиска уязвимостей в исходном коде ИИ-агента и зависимостях Python/Docker до момента деплоя на стенд.

3.1.2. Функциональное назначение всей системы

Система активной сетевой защиты с ИИ (САЗИИ) предназначена для автоматизации процессов обнаружения, анализа, классификации и нейтрализации сетевых угроз в распределенных контейнеризированных средах, а также для проведения непрерывного аудита безопасности программного обеспечения на этапах его разработки и развертывания.

В соответствии с заложенной архитектурой, система выполняет следующие основные функции:

1. Функция раннего обнаружения и децепции (Deception & Early Detection):

- Создание ложного периметра безопасности (Honeynet), перенаправляющего и изолирующего попытки сканирования портов, разведки сети и подбора учетных данных (брутфорса), предпринимаемые злоумышленниками.

- Фиксация любых обращений к неавторизованным эндпоинтам и системным ловушкам без создания ложноположительных срабатываний от легитимных пользователей.

2. Функция непрерывного сбора и нормализации телеметрии (Log Aggregation & Normalization):

- Сбор журналов событий безопасности в реальном времени с хостов виртуализации, веб-шлюзов и контейнеризированных приложений.

- Приведение неструктурированных текстовых логов (доступ Nginx, сессии Cowrie, системные вызовы) к единому машинно-читаемому JSON-формату для последующей машинной обработки.

3. Функция интеллектуального анализа и триажа угроз (AI-Powered Triage & Contextual Analysis):

- Семантический анализ агрегированных логов с использованием языковых моделей (LLM) для определения намерений атакующего.

- Автоматическая классификация выявленных инцидентов в соответствии с международным классификатором угроз MITRE ATT&CK (определение используемых тактик и техник).

- Первичная фильтрация и подавление ложноположительных срабатываний (False Positives) за счет глубокого контекстного анализа.

4. Функция активного реагирования на инциденты (Automated Active Defense / SOAR):

- Принятие автоматизированных решений об изоляции источника угрозы без участия оператора (уменьшение показателя MTTR — Mean Time to Respond).

- Динамическое управление правилами межсетевого экрана хоста (iptables/nftables) для мгновенной блокировки сетевых соединений скомпрометированных IP-адресов.

- Автоматическая регистрация инцидентов, формирование карточек уязвимостей и выгрузка отчетов в централизованную систему учета (DefectDojo).

5. Функция обеспечения безопасности разработки (DevSecOps / Shift-Left Security):

- Автоматизированная проверка исходного кода ИИ-агента на уязвимости (статический анализ SAST через Semgrep).

- Сканирование сторонних библиотек и контейнерных образов на наличие известных уязвимостей CVE (анализ SCA через Trivy) на каждом этапе сборки в конвейере GitHub Actions.

- Контроль неизменяемости и безопасности конфигураций инфраструктуры перед ее развертыванием на тестовом стенде.

7. Функция защиты от состязательных атак (Adversarial Robustness / Prompt Guard):

- Защита самого когнитивного ядра ИИ-агента от попыток манипуляции через вредоносные внедрения в поступающих логах (защита от атак Prompt Injection).

3.2. Сведения о выполнении требований ИБ

В процессе проектирования и реализации Системы Активной Сетевой Защиты с ИИ (САЗИИ) были учтены современные требования стандартов в области информационной безопасности (включая принципы ISO/IEC 27001, NIST Cybersecurity Framework и OWASP API Security Top 10).

Выполнение требований ИБ обеспечивается за счет реализации следующих принципов и защитных механизмов:

1. Реализация принципа наименьших привилегий (Principle of Least Privilege — PoLP):

- Все компоненты системы (микросервисы, ИИ-агент, сборщик логов) функционируют внутри изолированных контейнеров Docker под управлением непривилегированных учетных записей (не от имени суперпользователя root).
- Доступ к управлению межсетевым экраном хоста (iptables) ограничен строго изолированным системным контекстом ИИ-агента с использованием минимально необходимых разрешений.

2. Обеспечение целостности и конфиденциальности данных (Data Integrity & Confidentiality):

- Передача логов и телеметрии между компонентами внутри инфраструктуры защищена на уровне виртуальных сетей Docker.
- Конфиденциальные параметры (API-ключи языковых моделей, учетные данные интеграций) не хранятся в открытом виде в исходном коде, а передаются через защищенные переменные окружения (Environment Variables) и секреты среды выполнения GitHub Actions.

3. Обеспечение непрерывности и отказоустойчивости (Availability & Resilience):

- Архитектура системы построена по принципу разделения сред: компрометация ловушек (Honeynet) не приводит к нарушению работоспособности боевого сегмента (Production Zone).
- Использование автоматизированного перезапуска контейнеров (Restart Policies) минимизирует время простоя системы при аппаратных или программных сбоях.

4. Защита от состязательных атак на когнитивное ядро (AI Resilience):

- Внедрение модуля предварительной фильтрации (Prompt Guard) обеспечивает защиту ИИ-агента от деструктивного воздействия вредоносных

инструкций (Prompt Injection), передаваемых через сетевые логи, что гарантирует стабильность и предсказуемость принимаемых автоматизированных решений.

5. Обеспечение прозрачности и подотчетности (Accountability & Auditing):

- Каждый этап жизненного цикла кода фиксируется в системе контроля версий (GitHub) с обязательным прохождением статического анализа (SAST) и сканирования уязвимостей (SCA).
- Все инциденты безопасности, классифицированные ИИ-агентом, автоматически документируются и передаются в систему учета (DefectDojo), формируя неизменяемый аудиторский след.

3.2.1. Сведения об используемых средствах защиты информации

Для реализации функций обнаружения, анализа, предотвращения угроз и обеспечения безопасности процесса разработки в составе САЗИИ применяется комплекс следующих средств защиты информации (СЗИ) и программных инструментов:

1. Средства децепции и раннего обнаружения атак (Deception Technology):

- Cowrie SSH/Telnet Honeypot: Программно-эмулируемая ловушка для перехвата атак подбора паролей (брутфорса) и анализа вредоносных команд, вводимых злоумышленниками.
- Мониторинг сетевых точек входа (Nginx Reverse Proxy): Средство перенаправления и логирования входящих запросов к API.

2. Средства межсетевого экранирования и фильтрации трафика (Network Filtering):

- Linux Netfilter / iptables: Встроенный в ядро ОС межсетевой экран с поддержкой контроля состояний (Stateful Firewall), используемый модулем

активного реагирования для динамической блокировки IP-адресов нарушителей.

3. Средства анализа безопасности кода и зависимостей (Application Security Testing — AST):

- Semgrep: Инструмент статического анализа кода (SAST), выполняющий поиск уязвимостей и ошибок проектирования в исходном коде ИИ-агента и микросервисов на этапе сборки.
- Trivy: Сканер уязвимостей (SCA) для проверки базовых образов контейнеров и сторонних программных библиотек на наличие известных уязвимостей (CVE).

4. Средства централизованного учета уязвимостей и инцидентов (Vulnerability Management):

- DefectDojo: Платформа управления уязвимостями с открытым исходным кодом, используемая для агрегации находок, формирования тикетов безопасности и ведения реестра инцидентов.

5. Средства автоматизации и оркестрации (CI/CD & Infrastructure as Code):

- GitHub Actions: Платформа непрерывной интеграции и доставки, обеспечивающая автоматизированный запуск тестов, проверок безопасности и развертывание стенда.
- Docker & Docker Compose: Средства контейнеризации и оркестрации локального окружения, обеспечивающие изоляцию компонентов системы на уровне ядра ОС.

6. Средства защиты когнитивного ядра (LLM Guardrails):

- Prompt Guard (Встроенный фильтр регулярных выражений и семантической валидации): Программный модуль предварительной обработки текста, защищающий языковую модель от атак типа Prompt Injection.

3.2.2. Матрица ролей и полномочий

В соответствии с принципом разделения обязанностей (Segregation of Duties — SoD) и моделью управления доступом на основе ролей (RBAC), в Системе Активной Сетевой Защиты с ИИ (САЗИИ) разграничены права доступа для пользователей и служебных сервисов.

Ниже приведены три нормативные таблицы, регламентирующие ролевую модель системы:

Таблица 1 — Описание ролей и полномочий

Наименование роли	Назначение и зона ответственности	Ключевые полномочия
Администратор безопасности (SecOps)	Полный контроль над политиками безопасности, моделями ИИ-агента и правилами реагирования	- Настройка промптов и параметров модуля Prompt Guard - Управление правами доступа в DefectDojo - Ручная разблокировка ложно заблокированных IP-адресов - Утверждение исключений безопасности (Security Exceptions).
DevOps-инженер (Инфраструктура)	Управление средой развертывания, контейнеризацией и конвейером CI/CD	- Настройка пайплайнов в GitHub Actions - Администрирование Docker-окружения и виртуальных сетей. - Развертывание и сопровождение агентов Fluent Bit - Управление базовыми образами контейнеров.
Разработчик сервисов (Developer)	Разработка исходного кода микросервисов и API-эндпоинтов	- Создание веток и коммитов в репозитории GitHub - Просмотр отчетов статического анализа (SAST/SCA) в Pull Request - Исправление найденных уязвимостей в коде приложений
ИИ-агент (Служебная роль / Service Principal)	Автономный компонент системы, выполняющий функции анализа и активного реагирования	- Чтение нормализованных логов от Fluent Bit - Исполнение команд добавления блокирующих правил в iptables

		- Создание тикетов и выгрузка отчетов об инцидентах в DefectDojo через API - Запись собственных служебных метрик и логов.
Аудитор безопасности (Auditor / Read-Only)	Независимый контроль соответствия процессов и анализ защищенности	- Просмотр журналов аудита, дашбордов и отчетов в DefectDojo в режиме «только чтение» - Выгрузка отчетов соответствия комплаенсу. - Контроль неизменяемости логов.

Таблица 2 — Матрица доступа к объектам системы

Объект / Ресурс системы	Администратор безопасности (SecOps)	DevOps-инженер	Разработчик	ИИ-агент (Сервис)	Аудитор ИБ
Конфигурация фаервола (iptables / nftables)	R, U, D	R	—	C, E	R
Когнитивное ядро и промпты ИИ-агента	C, R, U, D	R	—	R, E	R
Система учета уязвимостей (DefectDojo)	C, R, U, D	R	R	C, R	R
Репозиторий кода и CI/CD (GitHub Actions)	R, U	C, R, U, E	C, R (PR)	—	R
Конфигурация сбора логов (Fluent Bit)	R, U	C, R, U, E	—	—	R
Сырые логи и журналы аудита	R	R	—	R	R
Исходный код	R	R	C, R, U	—	R

целевых микросервисов					
--------------------------	--	--	--	--	--

Условные обозначения прав доступа:

- C (Create) — Создание объекта / добавление записи.
- R (Read) — Чтение / просмотр информации.
- U (Update) — Модификация / редактирование параметров.
- D (Delete) — Удаление объекта.
- E (Execute) — Исполнение команд / запуск скрипта.
- « — » — Доступ отсутствует (запрещено).

Таблица 3 — Матрица конфликтов ролей (Segregation of Duties)

Роль 1	Роль 2	Уровень конфликта	Обоснование конфликта и риски
Разработчик	Администратор безопасности	Критический	Разработчик может отключить проверки SAST/SCA в CI/CD или изменить промпты ИИ-агента, скрыв уязвимости собственного кода от аудита
Разработчик	DevOps-инженер	Высокий	Риск прямого неконтролируемого деплоя непроверенного кода в продуктивную среду в обход правил защиты веток (Branch Protection)
Администратор безопасности	Аудитор ИБ	Высокий	Нарушение независимости аудита: администратор не может объективно оценивать эффективность настроенных им самим правил безопасности
DevOps-инженер	Аудитор ИБ	Средний	Риск сокрытия сбоев конфигурации

			инфраструктуры и манипуляции метриками доступности
ИИ-агент (Служебная роль)	Любая пользовательская роль	Критический	Учетные данные ИИ-агента (сервисные токены) не должны использоваться людьми для предотвращения компрометации служебных API-ключей

3.2.3. Решение по обеспечению отказоустойчивости

Отказоустойчивость Системы Активной Сетевой Защиты с ИИ (САЗИИ) обеспечивается комплексом архитектурных, программных и сетевых решений, гарантирующих непрерывность выполнения критических функций безопасности даже при сбоях отдельных компонентов, сетевых перегрузках или недоступности внешних API языковых моделей.

Архитектура отказоустойчивости системы разделена на следующие ключевые уровни:

УРОВНИ ОТКАЗОУСТОЙЧИВОСТИ САЗИИ	
1. Контейнерная среда (Docker Engine)	- Авто-перезапуск (Restart Policies) - Проверки здоровья (Healthchecks)
2. Сбор телеметрии (Fluent Bit)	- Дисковая буферизация (Disk Buffering) - Защита от переполнения (Backpressure)
3. Когнитивное ядро (ИИ-агент)	- Паттерн «Предохранитель» (Circuit Breaker) - Fallback на эвристические правила
4. Сетевой экран (iptables / Ядро Linux)	- Независимость Netfilter от процесса агента

	- Сохранение правил в памяти ядра ОС
5. Сохранность данных (DefectDojo / БД)	- Персистентные тома (Docker Named Volumes) - Регулярные дампы базы данных

1. Отказоустойчивость контейнерной среды и изоляция сбоев (Failure Isolation)

- **Политики автоматического перезапуска:** Для всех критически важных сервисов (ИИ-агент, Fluent Bit, Nginx, база данных DefectDojo) в манифестах docker-compose.yml задана директива restart: always (или unless-stopped). В случае аварийного завершения процесса (OOM, сбой памяти) Docker автоматически перезапускает упавший контейнер за доли секунды.

- **Проверки жизнеспособности (Healthchecks):** Контейнеры оснащены встроенными скриптами проверки доступности (HEALTHCHECK). При зависании процесса статус контейнера переходит в unhealthy, инициируя процедуру автоматического восстановления.

- **Изоляция сред:** Падение или перегрузка ловушек (Honeynet) физически не влияет на работу боевого сегмента (Production Zone) благодаря изоляции сетевых пространств имен (Network Namespaces) и квот ресурсов (cgroups).

2. Отказоустойчивость подсистемы сбора логов (Fluent Bit Resilience)

- **Буферизация на диск (Storage & Disk Buffering):** Для предотвращения потери сетевых событий при резких всплесках атак (DDoS/сканирование) во Fluent Bit включен механизм дискового буфера (storage.type filesystem). Логи сначала сохраняются на диск и только потом передаются ИИ-агенту.

- **Контроль обратного давления (Backpressure Management):**

Если ИИ-агент временно не успевает обрабатывать входящие события, Fluent Bit приостанавливает вычитку новых логов из сокетов без падения службы и без переполнения оперативной памяти хоста.

3. Отказоустойчивость когнитивного ядра ИИ-агента (Fallback & Circuit Breaker)

- **Таймауты и обработка сетевых ошибок:** При взаимодействии с внешними/локальными API языковых моделей (LLM API) реализованы жесткие таймауты (Connect/Read Timeouts не более 3–5 секунд).

- **Резервный режим анализа (Fallback to Heuristics / Circuit Breaker):**

- Если внешний API языковой модели становится недоступен или превышены квоты запросов, ИИ-агент автоматически активирует паттерн Circuit Breaker и переключается на встроенный эвристический движок (анализ логов на основе сигнатурных регулярных выражений и пороговых счетчиков попыток входа).

- Это гарантирует, что система продолжит блокировать явные сетевые атаки даже при полном отключении нейросетевого модуля.

- **Асинхронная очередь задач (Async Queue):** Обработка логов в Python реализована через асинхронные очереди (asyncio.Queue), что исключает блокировку сетевого шлюза во время ожидания генерации ответа от LLM.

4. Безопасный отказ сетевого контура (Fail-Safe Network State)

- **Автономия правил фаервола:** Правила блокировки (iptables -I INPUT -s <IP> -j DROP), добавленные ИИ-агентом, записываются непосредственно в таблицы ядра Linux (Netfilter).

- Если сам сервис ИИ-агента будет остановлен или перезагружен, все ранее заблокированные IP-адреса злоумышленников останутся

заблокированными на уровне ядра ОС, предотвращая несанкционированный доступ в момент перезапуска аналитического модуля.

5. Сохранность данных и персистентность (Data Persistence)

- **Выделенные тома (Docker Volumes):** База данных системы учета уязвимостей DefectDojo (PostgreSQL) и сессионные дампы ловушки Cowrie подключены через персистентные именованные тома (volumes:). Уничтожение или обновление образов контейнеров не приводит к потере исторических отчетов, метрик и аудиторского следа.

6. Отказоустойчивость процессов доставки в CI/CD (GitHub Actions)

- Конвейер сборки и деплоя спроектирован по принципу атомарности: при непрохождении тестов безопасности (SAST/SCA) или ошибках деплоя на этапе chaos-verify новая версия системы не вводится в эксплуатацию, а рабочий стенд сохраняет предыдущее стабильное состояние (Rollback).